

Home Cybersecurity Lab

Build Log, Architecture, and Detection Engineering Portfolio

Personal home lab — hypervisor, SIEM, Active Directory, and attacker infrastructure built from scratch on personal hardware.

1. Executive Summary

This document records the build of a three-node home cybersecurity lab designed to gain hands-on, resume-relevant experience with SIEM administration, Active Directory, endpoint telemetry, attack simulation, and detection engineering. The lab was built entirely on personal hardware: three laptops (16GB RAM each) and one desktop (8GB RAM, used only for management/browsing).

The lab includes a bare-metal Proxmox hypervisor running Security Onion (Suricata, Zeek, Elastic Stack), a VirtualBox-based Active Directory environment (a Domain Controller and a domain-joined Windows client, both instrumented with Sysmon and Elastic Agent), and a Kali Linux attacker machine. Over the course of the build, a wide range of real infrastructure problems were diagnosed and resolved — networking, firewall configuration, certificate trust, DNS, and Windows logging gaps — and the resulting environment was used to simulate and detect multiple real-world attack techniques, culminating in three custom, validated Sigma detection rules.

2. Lab Architecture

The lab is spread across three physical machines, each running a hypervisor, connected through a shared home Wi-Fi network (192.168.123.0/24). A private, isolated internal network (10.10.10.0/24) additionally connects the two Active Directory machines.

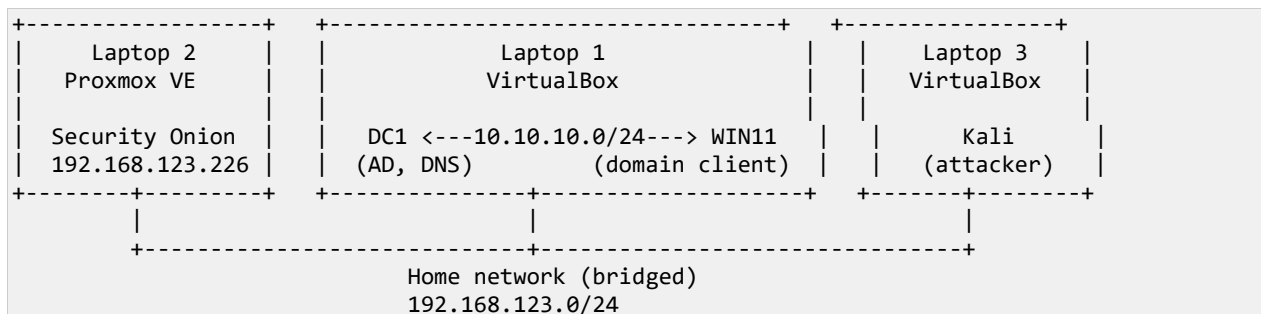
2.1 Physical / Virtual Layout

Machine	Hypervisor	Role	VM(s)	Key IP(s)
Laptop 2	Proxmox VE (bare metal)	Detection / SIEM server	Security Onion	192.168.123.226 (mgmt)
Laptop 1	VirtualBox (hosted)	AD victim environment	DC1 (Domain Controller) WIN11 (domain client)	DC1: 10.10.10.10 / 192.168.123.x WIN11: 10.10.10.20 / 192.168.123.x
Laptop 3	VirtualBox (hosted)	Attacker machine	Kali Linux	192.168.123.247
Desktop	—	Management / browsing	—	—

2.2 Network Segments

- Bridged home network (192.168.123.0/24) — every machine's “WAN-facing” adapter connects here, since it's the only path devices on different hypervisors (Proxmox vs. VirtualBox) can reach each other.
- Internal AD network (10.10.10.0/24, VirtualBox “Internal Network”) — an isolated virtual switch used only by DC1 and WIN11. WIN11's DNS is configured to DC1's internal IP (10.10.10.10), so this segment remains load-bearing for domain authentication even though a second path now exists.
- Security Onion monitor interface (ens19) — deliberately left unconnected to anything external; it exists to sniff traffic on Proxmox's own virtual switch only.

2.3 Text Diagram



3. Build Process

3.1 Proxmox + Security Onion

- Installed Proxmox VE bare-metal on Laptop 2; disabled the paid enterprise/Ceph repositories and enabled the free pve-no-subscription repo.
- Verified real available disk (local-lvm, ~374GB) versus the smaller root filesystem (local, ~94GB) shown on the summary page — VM disks draw from local-lvm.
- Deployed Security Onion (Eval Mode) as a VM: 4 vCPU, 8GB RAM, 150GB disk, two virtual NICs (ens18 = management, ens19 = monitor).
- Configured static management IP (192.168.123.226/24), gateway, and DNS during the Security Onion setup wizard.

3.2 Active Directory Environment

- Installed VirtualBox on Laptop 1; created DC1 (Windows Server, 4GB RAM, 2 vCPU, 60GB disk) and promoted it to a Domain Controller for a new forest, lab.local.
- Created WIN11 (Windows 11, 5GB RAM, 3 vCPU, 65GB disk), set a static IP on the 10.10.10.0/24 segment pointing DNS at DC1, and joined it to the lab.local domain.
- Installed Sysmon (SwiftOnSecurity configuration) on both DC1 and WIN11 for high-fidelity process, network, and file telemetry.
- Added a second, bridged network adapter to both DC1 and WIN11 so they could reach Security Onion on the home network while keeping the internal AD segment intact.

- Enrolled both machines in Security Onion's Elastic Fleet, with the Windows integration configured to collect Sysmon Operational, PowerShell Operational, and Security event log channels.

3.3 Kali Attacker Machine

- Imported Kali Linux's official pre-built VirtualBox image on Laptop 3 (8GB RAM, 4 vCPU, single bridged adapter).
- Verified connectivity to Security Onion, DC1, and WIN11 over the bridged home network, and updated the system (apt update && apt upgrade).
- Installed Atomic Red Team (PowerShell-based MITRE ATT&CK test framework) on DC1 for structured technique testing.

4. Troubleshooting Journal

The build surfaced a long series of real infrastructure problems. Each is recorded below with root cause and resolution, since diagnosing them was as valuable as the end result.

Issue	Root Cause	Resolution
Proxmox black screen / no Ethernet	Wi-Fi-only laptop; installer needs a NIC driver at install time	Selected the detected Wi-Fi NIC as the management interface and supplied Wi-Fi credentials during install
Security Onion setup: bonding failure on monitor NIC	bond0 / bond0-slave-ens19 configured for MTU 9000 (jumbo frames); VirtIO NIC doesn't support it	Set both bond0 and the slave interface to MTU 1500 via nmcli, then cycled the connections
Security Onion setup: "IP being routed is not the IP assigned"	ens19 held a leftover DHCP lease and a competing default route with a lower metric than ens18	Flushed the IP from ens19 and brought it down before re-running so-setup
Elastic Agent download failed (empty reply, 404)	/opt/socore/html (nginx's static artifact directory) was never created — a genuine provisioning gap in this install	Workaround: downloaded the Elastic Agent installer directly from Elastic's site instead of Security Onion's local artifact server

Elastic Agent enrollment: x509 certificate error	Fleet Server uses a self-signed certificate	Added --insecure to the install/enroll command
Agent healthy in Fleet but shipping no data	DNS: agents resolved "securityonion" via public DNS (8.8.8.8), which has no record for it	Added a manual hosts file entry (192.168.123.226 securityonion) on each Windows machine
Elasticsearch output degraded (x509 error, separate from enrollment)	The --insecure flag only covered the enrollment connection, not the ongoing Elasticsearch output	Set ssl.verification_mode: none on the so-manager_elasticsearch Fleet output
Elasticsearch (port 9200) unreachable from agents	Docker/iptables only allowed traffic to port 9200 from Security Onion's own internal Docker network and itself — external hosts were never permitted	Located the correct hostgroup (elasticsearch_rest) not exposed in the web UI; added it directly via the pillar file (soc_firewall.sls) and re-ran a highstate
Suricata/Zee k show zero traffic between Kali and DC1	Architectural: Proxmox connects over Wi-Fi, and both Kali and DC1 reach each other directly through the home router. That traffic never crosses Proxmox's virtual switch, so Security Onion's monitor interface has no visibility into it	Documented as a known limitation; relied on host-based telemetry (Sysmon, Windows Event Logs) for detection instead

PowerShell encoded command produced no Event ID 4104 logs	PowerShell Script Block Logging was not enabled by default on the Windows machine	Enabled EnableScriptBlockLogging via registry policy (HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging)
Sigma rule failed to convert (HTTP 500)	Security Onion's Sigma-to-EQL converter did not support the count() by field > N aggregation syntax used for a brute-force threshold rule	Rewrote the rule as a simple field-match (single-event) detection instead of a threshold-based one

5. Attack Simulation and Detection Results

Each scenario below was executed from Kali or directly on the target Windows machine, then confirmed against real ingested data in Security Onion.

Scenario	Tool / Technique	MITRE ATT&CK	Detected Via	Result
Network reconnaissance	Nmap (-sV -A)	T1046 – Network Service Discovery	—	Traffic reached the target; not visible to Suricata/Zeek (Wi-Fi visibility gap). No Sysmon Event ID 3 generated — scan traffic didn't meet Sysmon's logging threshold.
Authenticated file share access	smbclient	T1021.002 – SMB/Windows Admin Shares	Windows Security Event 4624 (Logon Type 3)	Successful logon captured, including source IP and account name
Brute-force / password guessing	netexec	T1110 – Brute Force	Windows Security Event 4625	Multiple failed logon events captured, same account and source IP

Credential dumping attempt	Mimikatz (sekurlsa::logonpasswords, sekurlsa::tickets)	T1003 – OS Credential Dumping	Sysmon Event ID 1 (process creation)	Process launch logged; extraction itself blocked by modern Windows LSA Protection
Obfuscated command execution	Encoded PowerShell (- EncodedCommand)	T1059.001, T1027 – PowerShell / Obfuscation	PowerShell Operational Event ID 4104	Not visible until Script Block Logging was enabled; fully visible (decoded) afterward
Structured technique test	Atomic Red Team T1082	T1082 – System Information Discovery	Windows Defender detection log	Simulated technique blocked and logged by Windows Defender

6. Custom Detection Rules (Sigma)

Three Sigma detection rules were authored, converted to EQL, enabled, and validated against live attack traffic generated in this lab.

6.1 Failed Logon Attempt Detected

Detects a failed Windows logon (Event ID 4625), the base signal for brute-force / password-guessing activity.

```

title: 'Failed Logon Attempt Detected'
id: 7d8f3a1e-4b2c-4e9a-9f1d-2a3b4c5d6e7f
status: 'experimental'
description: |
    Detects a failed logon attempt (Event ID 4625) which may indicate an
    incorrect password or an attempted brute-force attack against a
    Windows account.
author: '@user'
tags:
  - attack.credential_access
  - attack.t1110
logsource:
  category: authentication
  product: windows
detection:
  selection:
    EventID: 4625
  condition: selection
level: 'medium'

```

Note: an initial version used condition: `selection | count() by IpAddress > 3` to require multiple failures from one source before alerting. This aggregation syntax was not supported by Security Onion's Sigma-to-EQL converter (HTTP 500 on conversion) and was replaced with the simple single-event version above. A proper threshold-based version of this rule would be implemented via ElastAlert rather than Sigma.

6.2 Mimikatz Process Execution Detected

Detects execution of mimikatz.exe by filename, regardless of the directory it's launched from.

```
title: 'Mimikatz Process Execution Detected'
id: 3f4a5b6c-7d8e-4f9a-b0c1-d2e3f4a5b6c7
status: 'experimental'
description: |
    Detects execution of mimikatz.exe, a well-known credential dumping
    tool used to extract passwords, hashes, and Kerberos tickets from
    memory. Execution of this tool is a strong indicator of credential
    access activity and is rarely, if ever, legitimate.
author: '@user'
tags:
  - attack.credential_access
  - attack.t1003
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    Image|endswith: '\\mimikatz.exe'
  condition: selection
level: 'high'
```

6.3 Encoded PowerShell Command Execution

Detects PowerShell invoked with the `-EncodedCommand` / `-enc` flag, a common technique for hiding command intent and evading simple text-based detection.

```
title: 'Encoded PowerShell Command Execution'
id: 8a9b0c1d-2e3f-4a5b-6c7d-8e9f0a1b2c3d
status: 'experimental'
description: |
    Detects execution of PowerShell with the -EncodedCommand (or
    shorthand -enc/-e) parameter, commonly used by attackers to
    obfuscate malicious commands and evade simple text-based
    detection. Legitimate administrative use of this flag is uncommon.
author: '@user'
tags:
  - attack.execution
  - attack.t1059.001
  - attack.defense_evasion
  - attack.t1027
logsource:
  category: process_creation
  product: windows
detection:
  selection_img:
    - Image|endswith: '\\powershell.exe'
    - OriginalFileName: 'PowerShell.EXE'
  selection_cli:
```

```
CommandLine|contains|windash:
- '-enc'
- '-EncodedCommand'
condition: all of selection_*
level: 'high'
```

7. Known Limitations

- Network-based detection (Suricata/Zeek) has no visibility into traffic between other bridged devices (e.g., Kali → DC1). Proxmox connects to the home network over Wi-Fi, and Wi-Fi/consumer-router traffic between two other clients never passes through Proxmox's own network stack, so it can't be mirrored to the monitor interface. Wired switches have the same limitation unless the switch specifically supports a SPAN/mirror port, which consumer routers do not.
- The Security Onion nginx artifact directory (/opt/socore/html) was never created by this install; the Fleet-provided one-line agent download command fails as a result. Manual download from Elastic's site is used as a permanent workaround.
- Suricata shows a persistent “rule mismatch” status under Detections. The documented cause (Elasticsearch disk watermark) was checked and ruled out (/nsm at 19% used). Not pursued further, since it does not affect any currently-used detection path (Sigma/EQL).
- Sysmon (SwiftOnSecurity config) did not log the Nmap scan or the initial SMB connection as Event ID 3 (network connection) — the default config is tuned toward outbound/anomalous connections rather than every inbound connection to a standard service.

8. Lessons Learned

- Defense in depth is real, not just a slogan: three different detection layers (network, host telemetry, endpoint protection) each caught different things, and each also had blind spots the others covered.
- Logging is not “on” by default — PowerShell Script Block Logging had to be explicitly enabled before encoded commands became visible, which is also true in many real enterprise environments.
- Being on the same subnet does not mean a monitoring tool can see all traffic on that subnet; genuine network visibility requires being in the physical/virtual path the traffic actually flows through (e.g., a SPAN port, or a shared hypervisor virtual switch).
- Splitting a lab across multiple hypervisors on separate physical machines is beginner-friendly and resource-flexible, but trades away easy network-level visibility — consolidating everything onto one hypervisor's virtual network would resolve this at the cost of needing significantly more RAM (~32GB) on a single machine.
- Detection rule syntax support varies by platform — a Sigma rule using aggregation logic that's valid Sigma still needs to be checked against what the specific backend (Security Onion's Sigma-to-EQL converter, in this case) actually supports.

9. Future Work

- Consolidate the lab onto a single hypervisor (or add a managed switch with SPAN/mirror support) to restore genuine network-based detection.

- Build ElastAlert-based threshold rules (e.g., true “N failed logons in 5 minutes” brute-force detection) to complement the simpler Sigma rules.
- Add Splunk and/or Microsoft Sentinel as a second and third SIEM to broaden tool coverage.
- Expand Atomic Red Team coverage across additional MITRE ATT&CK tactics (lateral movement, persistence, exfiltration).
- Investigate and resolve the Suricata rule-mismatch status once network visibility is restored.